Science-Bitch

Complete Scientific Method Workflow Tool

WAFT Research Team

waft@example.com

ABSTRACT

A comprehensive guide to the Science-Bitch command-line tool that implements the complete scientific method workflow for systematic hypothesis testing, controlled experimentation, and evidence-based conclusions.

2026-01-14

1. Science-Bitch: Complete Command Reference & Guide

Version: 1.0
Date: 2026-01-14
Author: WAFT Research Team

1.1 Table of Contents

1. Overview
2. Quick Start
3. Complete Workflow Sequence
4. Command Options
5. Usage Examples
6. Scientific Method Cycle
7. File Structure
8. Integration
9. Best Practices
10. Troubleshooting
11. Technical Implementation
12. Related Commands

1.2 Overview

Science-Bitch is a comprehensive command-line tool that implements the complete scientific method workflow for systematic hypothesis testing, controlled experimentation, and evidence-based conclusions.

Purpose

This command provides:
- **Complete Scientific Method**: Full workflow from hypothesis to conclusion
- **State Capture**: Initial state (A) and final state (B) tracking
- **Data Collection**: Systematic data collection during experiments (C)
- **Hypothesis Testing**: Form, test, and verify hypotheses
- **Report Generation**: Generate field guides and project status reports
- **Evidence-Based Analysis**: Systematic analysis with traceable evidence

Key Features

- **Complete Scientific Method Implementation**: Full workflow from hypothesis formation to conclusion
- **State Capture System**: Automatic tracking of system states before (A) and after (B) experiments
- **Systematic Data Collection**: Comprehensive data collection during experiments (C)
- **Hypothesis Testing Framework**: Structured approach to forming, testing, and verifying hypotheses
- **Automated Report Generation**: Professional PDF reports with full documentation
- **Evidence-Based Analysis**: Systematic analysis with traceable evidence chains

1.3 Quick Start

Interactive Workflow

```
waft science-bitch
```

Runs the complete interactive scientific method workflow.

Generate Field Guide

```
waft science-bitch --field-guide
```

Generates comprehensive field guide PDF.

Generate Project Status Report

```
waft science-bitch --report
```

Generates project status report PDF.

With Hypothesis

```
waft science-bitch --hypothesis "Higher skill i
```

Starts workflow with a pre-defined hypothesis.

Run Experiment

```
waft science-bitch --run
```

Runs experiment directly.

1.4 Complete Workflow Sequence

Phase 1: Form Hypothesis

Purpose: Create a testable hypothesis

Process:
1. Identify the question or phenomenon to investigate
2. Formulate a clear, testable hypothesis
3. Define variables (independent, dependent, control)
4. Specify what would verify or refute the hypothesis

Output: Hypothesis object with statement, variables, and verification plan

Example:

```
Hypothesis(
    statement="Increasing iteration count improv
    prediction="Fitness scores will increase by
    variables=[
        Variable("iteration_count", VariableType
        Variable("fitness_score", VariableType.D
        Variable("random_seed", VariableType.CON
    ]
)
```

Phase 2: Design Experiment

Purpose: Create experiment design based on hypothesis

Process:
1. Define experiment structure
2. Specify variables and their types
3. Design data collection methods
4. Plan state capture points (A and B)
5. Define success criteria

Output: Experiment design with variables, data collection plan, and state capture strategy

Key Components:
- **Independent Variables**: What you're changing
- **Dependent Variables**: What you're measuring
- **Control Variables**: What stays constant
- **Data Collection Plan**: When and how to collect data
- **State Capture Points**: When to capture system state

Phase 3: Capture Initial State (A)

Purpose: Capture system state before experiment

Process:
1. Identify components to track
2. Capture initial state snapshot
3. Generate state hash for comparison
4. Save state to `_science/experiments/[exp_id]/state_a.json`

Output: Initial state snapshot with hash

Components Captured:
- System configuration
- Current metrics
- Environment state
- Resource usage
- Any relevant system state

Example State:

```
{
  "state_id": "state_a",
  "timestamp": "2026-01-14T19:44:33Z",
  "state_hash": "abc123...",
  "components": {
    "fitness_score": 0.65,
    "iteration_count": 0,
    "system_load": 0.3
  }
}
```

Phase 4: Run Experiment

Purpose: Execute experiment with data collection

Process:
1. Execute experiment according to design
2. Collect data points during execution (C)
3. Record measurements and observations
4. Track experiment progress
5. Handle errors and edge cases

Output: Experiment results with collected data

Execution Steps:
- Initialize experiment environment
- Set control variables
- Execute experiment logic
- Collect measurements at intervals
- Record observations
- Handle errors gracefully

Phase 5: Collect Data (C)

Purpose: Systematic data collection during experiment

Data Collected:
- Measurements at specified intervals
- Observations and notes
- Performance metrics
- State changes
- Error conditions
- Timestamps for all data points

Storage: Data saved to `_science/data/[exp_id]/`

Output: Data series with timestamps and metadata

Data Format:

```
{
  "data_series": {
    "fitness_scores": [
      {"timestamp": "2026-01-14T19:45:00Z", "v
      {"timestamp": "2026-01-14T19:46:00Z", "v
      {"timestamp": "2026-01-14T19:47:00Z", "v
    ],
    "iteration_counts": [
      {"timestamp": "2026-01-14T19:45:00Z", "v
      {"timestamp": "2026-01-14T19:46:00Z", "v
      {"timestamp": "2026-01-14T19:47:00Z", "v
    ]
  }
}
```

Phase 6: Capture Final State (B)

Purpose: Capture system state after experiment

Process:
1. Capture final state snapshot
2. Generate state hash for comparison
3. Compare with initial state (A)
4. Identify changes
5. Save state to `_science/experiments/[exp_id]/state_b.json`

Output: Final state snapshot with comparison to initial state

State Comparison:
- Compare component values
- Identify changes
- Calculate differences
- Document modifications

Example Comparison:

```
{
  "state_id": "state_b",
  "timestamp": "2026-01-14T19:50:00Z",
  "state_hash": "def456...",
  "components": {
    "fitness_score": 0.85,
    "iteration_count": 15,
    "system_load": 0.5
  },": 0.78}
  "changes_from_a": {
    "fitness_score": {"old": 0.65, "new": 0.85,
    "iteration_count": {"old": 0, "new": 15, "de
    "system_load": {"old": 0.3, "new": 0.5, "del
  }e": 15}
}
```

Phase 7: Analyze Results

Purpose: Analyze experiment data and verify/refute hypothesis

Analysis Performed:
1. Compare states A and B
2. Analyze collected data (C)
3. Verify or refute hypothesis
4. Calculate confidence level
5. Generate conclusions
6. Identify patterns and insights

Output: Analysis report with:
- Hypothesis verification status
- Confidence level
- Conclusions
- Recommendations
- Evidence summary

Analysis Example:

```
Analysis(
    verified=True,
    confidence=0.85,
    conclusions="Hypothesis verified: Increasing iterations from 5 to 15 improved fitness",
    evidence={
        "state_comparison": "Fitness score increased from 0.65 to 0.85",
        "data_trend": "Positive correlation between iterations and fitness",
        "statistical_significance": "p < 0.05"
    }
)
```

Phase 8: Generate Reports

Purpose: Create comprehensive documentation

1.5 Command Options

Interactive Workflow (Default)

```
waft science-bitch
```

Runs complete interactive workflow with prompts for each phase.

Report Types:

Field Guide

- Complete usage guide
- Workflow documentation
- Examples and best practices
- Command reference

Project Status Report

- Current project state
- Goals and objectives
- Evidence collected
- Next steps
- Progress tracking

Output: PDF reports in `_science/reports/`

What it does:
1. Prompts for hypothesis formation
2. Guides through experiment design
3. Captures initial state (A)
4. Runs experiment with data collection (C)
5. Captures final state (B)
6. Analyzes results
7. Generates report

Generate Field Guide

```
waft science-bitch --field-guide
```

Generates comprehensive field guide PDF showing how to use the command.

Output: `_science/reports/field_guide.pdf`

Content:
- Complete workflow documentation
- All command options
- Usage examples
- Best practices
- Troubleshooting

Generate Project Status Report

```
waft science-bitch --report
```

Generates project status report PDF with current state, goals, and progress.

Output: `_science/reports/project_status.pdf`

Content:
- Current project state
- Goals and objectives
- Evidence collected
- Next steps
- Progress tracking

With Pre-Defined Hypothesis

```
waft science-bitch --hypothesis "Your hypothesis statement here"
```

Starts workflow with a hypothesis already defined.

Example:

```
waft science-bitch --hypothesis "Increasing ite
```

What it does:
- Starts with pre-defined hypothesis
- Skips hypothesis formation phase
- Proceeds directly to experiment design
- Runs complete workflow

Run Experiment Directly

```
waft science-bitch --run
```

Runs experiment without interactive prompts (uses existing hypothesis/design).

Use when:
- Hypothesis and design already exist
- Want to re-run experiment
- Need to skip interactive prompts

Specify Project Path

```
waft science-bitch --path /path/to/project
```

Runs command in specified project directory (default: current directory).

1.6 Usage Examples

Example 1: Full Interactive Workflow

```
waft science-bitch
```

What it does:
1. Prompts for hypothesis formation
2. Guides through experiment design
3. Captures initial state (A)
4. Runs experiment with data collection (C)
5. Captures final state (B)
6. Analyzes results
7. Generates report

Output: Complete experiment with analysis and report

Example 2: Generate Field Guide

```
waft science-bitch --field-guide
```

What it does:
- Generates comprehensive field guide PDF
- Documents all workflow phases
- Provides examples and best practices
- Includes command reference

Output: `_science/reports/field_guide.pdf`

Example 3: Generate Status Report

```
waft science-bitch --report
```

What it does:
- Generates project status report PDF
- Shows current state and goals
- Documents evidence collected
- Lists next steps

Output: `_science/reports/project_status.pdf`

Example 4: Hypothesis-Driven Experiment

```
waft science-bitch --hypothesis "Increasing ite
```

What it does:
- Starts with pre-defined hypothesis
- Skips hypothesis formation phase
- Proceeds directly to experiment design
- Runs complete workflow

Output: Experiment results testing the specified hypothesis

1.7 Scientific Method Cycle

The command implements the complete scientific method:

1. **Observe**: Identify phenomenon or question
2. **Hypothesize**: Form testable hypothesis
3. **Design**: Create experiment with variables
4. **Capture State A**: Initial system state
5. **Run Experiment**: Execute with data collection
6. **Collect Data C**: Measurements during experiment
7. **Capture State B**: Final system state
8. **Analyze**: Verify/refute hypothesis
9. **Report**: Generate conclusions and documentation
10. **Iterate**: Modify variables and repeat

1.8 File Structure

```
_science/
├── README.md                    # Overview and quick start
├── SUMMARY.md                   # Implementation summary
├── experiments/                 # Experiment definitions and results
│   ├── [exp_id]/               # Individual experiment
│   │   ├── experiment.json     # Experiment definition
│   │   ├── state_a.json        # Initial state (A)
│   │   ├── state_b.json        # Final state (B)
│   │   └── results.json        # Experiment results
├── data/                        # Collected data (C)
│   └── [exp_id]/               # Data for each experiment
│       └── data_series.json   # Collected measurements
├── reports/                     # Generated reports
│   ├── field_guide.pdf         # Usage guide
│   ├── field_guide.md          # Source markdown
│   ├── project_status.pdf      # Status report
│   ├── project_status.md       # Source markdown
│   └── experiment_*.md         # Experiment reports
└── tools/                       # Helper utilities
    ├── generate_field_guide_pdf.py
    └── generate_status_pdf.py
```

1.9 Integration

This command integrates with:

- **Scientific Method Tool**: `scientific_method_tool/` provides core functionality
- **PDF Generation**: Uses `waft.evolution.pdf_generator` for reports
- **Work Efforts**: Tracks progress in `WE-260112-az3z`
- **State Capture**: Captures system states before/after experiments
- **Data Collection**: Records all measurements during experiments

Scientific Method Tool

Core functionality provided by `scientific_method_tool/`:

- Hypothesis formation and validation
- Experiment design
- State capture utilities
- Data collection framework
- Analysis tools

PDF Generation

Uses `waft.evolution.pdf_generator`:
- Professional formatting
- Clinical standard style
- Academic paper templates
- Automatic title extraction

Work Efforts

Tracks progress in work effort:
- `WE-260112-az3z_science_bitch_command_full_scientific_method_cli`
- Tickets and progress tracking
- Implementation documentation

1.10 Best Practices

1. **Clear Hypotheses**: Formulate specific, testable hypotheses
2. Be specific about what you're testing
3. Define clear success criteria
4. Specify variables explicitly
5. **Systematic Design**: Plan experiments carefully
6. Control for confounding variables
7. Design reproducible experiments
8. Plan data collection points
9. **Complete State Capture**: Capture all relevant components
10. Identify all system components
11. Capture comprehensive state
12. Generate state hashes for comparison
13. **Thorough Data Collection**: Collect data at appropriate intervals
14. Define measurement intervals
15. Record all observations
16. Include timestamps
17. **Careful Analysis**: Verify all claims with evidence
18. Compare states systematically
19. Analyze data trends
20. Calculate confidence levels
21. **Document Everything**: Generate reports for future reference
22. Create comprehensive reports
23. Include all evidence
24. Document conclusions
25. **Iterate**: Refine hypotheses and experiments based on results
26. Learn from each experiment
27. Refine hypotheses
28. Improve experimental design

1.11 Troubleshooting

Error: Command not found

Solution: Ensure you're in a project with `waft` installed
Check: `waft --help` should show `science-bitch` command

Error: ImportError

Solution: Ensure scientific_method_tool is available
Check: `python3 -c "from scientific_method_tool import Hypothesis"`

Error: Permission denied

Solution: Check file permissions on `_science/` directory
Fix: `chmod -R u+w _science/`

Error: State capture failed

Solution: Ensure components exist and are accessible
Check: Verify component paths and permissions

Error: PDF generation fails

Solution: Check that WeasyPrint is installed and working
Check: `python3 -c "import weasyprint"`

Error: Experiment data not found

Solution: Ensure experiment was run and data was collected
Check: Verify `_science/experiments/` and `_science/data/` directories

1.12 Technical Implementation

CLI Command

The command wraps the `waft science-bitch` CLI command:
- **Location**: `src/waft/main.py` (line 2144)
- **Manager**: `src/waft/core/science_bitch.py`
- **Work Effort**: `WE-260112-az3z`

Execution

When `/science-bitch` is executed:
1. AI reads this command file
2. Executes `waft science-bitch` with appropriate options
3. Processes results and displays output
4. Generates reports if requested

Options Mapping

- `/science-bitch` → `waft science-bitch` (interactive)
- `/science-bitch --field-guide` → `waft science-bitch --field-guide`
- `/science-bitch --report` → `waft science-bitch --report`
- `/science-bitch --hypothesis "..."` → `waft science-bitch --hypothesis "..."`
- `/science-bitch --run` → `waft science-bitch --run`

Manager Class

Location: `src/waft/core/science_bitch.py`

Key Methods:

- `run_interactive()` : Run full interactive workflow
- `generate_field_guide()` : Generate field guide PDF
- `generate_project_status_report()` : Generate status report PDF
- `_create_field_guide_content()` : Create field guide markdown
- `_create_project_status_content()` : Create status report markdown

1.13 Related Commands

- **`/hypothesis`** : Form and verify hypotheses (complementary workflow)
- **`/prove-it`** : Prove scientific method tool works
- **`/verify`** : Verify claims with evidence
- **`/analyze`** : Analyze data and generate insights
- **`/checkpoint`** : Document current state
- **`/reflect`** : Capture learnings and insights

1.14 When to Use

Use `/science-bitch` when:

- ✅ Need to test a hypothesis systematically
- ✅ Want to run controlled experiments
- ✅ Need evidence-based conclusions
- ✅ Want to track state changes (A → B)
- ✅ Need systematic data collection
- ✅ Want to generate scientific reports
- ✅ Need to verify or refute a hypothesis

Don't use `/science-bitch` when:

- ❌ Just need quick testing (use direct testing)
- ❌ Don't need systematic approach
- ❌ Hypothesis is already verified
- ❌ Just need data collection (use data tools directly)
- ❌ Time-constrained (full workflow takes time)

1.15 Output Summary

After completion, provides:
1. **Hypothesis**: Testable hypothesis statement
2. **Experiment Design**: Complete experiment structure
3. **State Snapshots**: Initial (A) and final (B) states
4. **Collected Data**: All measurements (C)
5. **Analysis**: Hypothesis verification/refutation with confidence
6. **Report**: Comprehensive documentation (if requested)

1.16 Conclusion

Science-Bitch provides a complete, systematic approach to scientific research and experimentation. By implementing the full scientific method workflow with automatic state capture, systematic data collection, and professional report generation, it enables rigorous, evidence-based research with full traceability.

Key Benefits:
- **Systematic Approach**: Structured workflow ensures completeness
- **Evidence-Based**: All conclusions traceable to data
- **Reproducible**: State capture enables reproduction
- **Professional**: High-quality documentation
- **Integrated**: Works with WAFT ecosystem

Getting Started:
1. Install dependencies: `pip install -e .`
2. Run field guide: `waft science-bitch --field-guide`
3. Form hypothesis: `waft science-bitch --hypothesis "Your hypothesis"`
4. Run experiment: `waft science-bitch --run`
5. Generate report: `waft science-bitch --report`

Resources:
- **Work Effort**: `WE-260112-az3z_science_bitch_command_full_scientific_method_cli`
- **Source Code**: `src/waft/core/science_bitch.py`
- **Documentation**: `_science/README.md`
- **Examples**: `scientific_method_tool/example_usage.py`

This guide provides complete documentation for the Science-Bitch command, covering all aspects from quick start through advanced usage, troubleshooting, and technical implementation.